LoAS+, Explained from Scratch

One small example, followed from start to finish: first through LoAS, then through each of the three changes that make LoAS+. Internal working notes, 18 October 2026.

Eight words used throughout. Spike train: what one input neuron does over the T timesteps, written as T bits (1 = spike), e.g. 0110 = spikes at t1 and t2. **Silent**: an input whose spike train is all zeros. **Weight**: one number of a filter. **Bitmask**: one bit per position, 1 where the value is non-zero (non-silent input, non-zero weight). **Fibre**: a bitmask plus its non-zero values packed one after another. **Chunk**: the part of a fibre one bitmask covers (128 positions in the real design). **Match**: a position where both the input is non-silent and the weight is non-zero. **TPPE**: LoAS's processing element; it computes one output neuron for all timesteps.

1. The problem

A spiking layer is simple arithmetic: for every output neuron and every timestep, add up the weights of the inputs that spiked. After pruning, most weights are 0 and most inputs are silent, so most of that work is about zeros. LoAS (Yin et al.) already handles this well in three ways:

- **All timesteps at once:** each TPPE keeps T sums, one per timestep, so T never multiplies the time.
- **Only non-silent inputs and non-zero weights are stored:** each is a fibre (bitmask + packed values), so zeros cost neither memory nor traffic.
- **Inner join:** the TPPE ANDs the two bitmasks to find the matches, and touches only those.

What LoAS still wastes is **time inside and between chunks**. To find where a match's values sit in the packed fibres it needs prefix sums (counting the 1s before a position). These are large circuits, so LoAS makes one fast and one slow ("laggy"). That forces it to guess the spike train (all ones) and correct the guess later, and it finds only one match per cycle. In addition, the 16 TPPEs move from chunk to chunk in lock step, each chunk starts with a fixed load and wait, and the slowest TPPE sets the pace. LoAS+ changes three things:

- **C1 Offsets without prefix sums** (Section 5): each fibre stores a small count at the start of every 16-position segment, so an offset is a lookup plus a count over 16 bits. No guess, no correction.
- **C2 Four matches per cycle** (Section 6): the chunk is split into four quarters, each with its own match finder.
- **C3 No waiting between chunks** (Section 7): empty weight chunks are skipped, the next chunk loads while the current one computes, and each TPPE runs at its own pace from a small queue.

Nothing is approximated. LoAS+ computes exactly the same sums and spikes as LoAS. It is about 3.4x faster on the same model, with 2.9-3.3x less energy (Section 10).

2. The example and the correct answer

We follow **one output neuron** (one filter at one output pixel) over one chunk of **16 inputs**, k0 to k15, with **T = 4** timesteps. The real design has 128 inputs per chunk, 32 per quarter, 16 per segment and T = 8; here everything is scaled down by 8 (16 inputs, 4 per quarter, 4 per segment). The colours mark the four quarters (k0-k3, k4-k7, k8-k11, k12-k15).

	k0	k1	k2	k3	k4	k5	k6	k7	k8	k9	k10	k11	k12	k13	k14	k15
spike train (t0 t1 t2 t3)	1010	0000	0110	0000	1111	0001	0000	0100	0000	0011	1000	0000	0101	0000	1100	0010
weight	0	4	3	0	2	0	0	1	0	-2	3	0	1	5	0	2

Figure 1. The chunk. Grey = silent input or zero weight. Outlined = a **match** (non-silent input and non-zero weight): k2, k4, k7, k9, k10, k12, k15. Only the matches change the result.

The answer. "3 x 0110" means: add 3 at every timestep whose bit is 1. Adding the 7 matches gives the output current [5,7,5,1] (one number per timestep):

input	weight	spike train	adds (t0..t3)
k2	3	0110	[0,3,3,0]
k4	2	1111	[2,2,2,2]
k7	1	0100	[0,1,0,0]
k9	-2	0011	[0,0,-2,-2]
k10	3	1000	[3,0,0,0]
k12	1	0101	[0,1,0,1]
k15	2	0010	[0,0,2,0]
		sum = output current	**[5,7,5,1]**

Figure 2. The output current of the neuron: [5,7,5,1]. Every design below must reproduce it.

The neuron. LoAS uses a leaky integrate-and-fire neuron with hard reset: V = current + carried; if V > 6 the neuron spikes and nothing is carried, otherwise half of V is carried to the next timestep. Here it spikes once, at t1, so its spike train is **0100**. This spike train goes into the next layer's spike fibre.

timestep	current	V before = current + carried	fires? (V > 6)	carried to next = 0.5 V
t0	5	5 + 0 = 5	no	2.5
t1	7	7 + 2.5 = 9.5	yes	0 (reset)
t2	5	5 + 0 = 5	no	2.5
t3	1	1 + 2.5 = 3.5	no	1.75

Figure 3. The neuron turns the current [5,7,5,1] into the spike train 0100. It runs once per output neuron, after its last chunk.

3. How LoAS stores a chunk

LoAS never stores zeros. For the spikes (fibre A) it keeps a bitmask with 1 for every non-silent input, followed by the non-silent spike trains packed one after another. All T bits of a train are kept together, so one read gets all timesteps. The weights (fibre B) work the same way: a bitmask, then the non-zero weights.

position	k0	k1	k2	k3	k4	k5	k6	k7	k8	k9	k10	k11	k12	k13	k14	k15
bitmask A	1	0	1	0	1	1	0	1	0	1	1	0	1	0	1	1
bitmask B	0	1	1	0	1	0	0	1	0	1	1	0	1	1	0	1

packed spike trains (fibre A)	**0**: k0	**1**: k2	**2**: k4	**3**: k5	**4**: k7	**5**: k9	**6**: k10	**7**: k12	**8**: k14	**9**: k15
value	1010	0110	1111	0001	0100	0011	1000	0101	1100	0010

packed weights (fibre B)	**0**: k1	**1**: k2	**2**: k4	**3**: k7	**4**: k9	**5**: k10	**6**: k12	**7**: k13	**8**: k15
value	4	3	2	1	-2	3	1	5	2

Figure 4. The chunk in memory. Fibre A holds 10 trains instead of 16, and fibre B holds 9 weights instead of 16. The small bold number is the **offset**: where a value sits in its packed list.

The catch: the packed lists have no gaps, so position k9 is not at offset 9. Its weight is at offset 4 of fibre B (four non-zero weights come before it), and its spike train is at offset 5 of fibre A. To find an offset you have to **count the 1s in the bitmask before that position**. That count is a prefix sum, and how fast you can do it decides how fast a TPPE runs.

4. How LoAS computes the chunk

4.1 Find the matches. The TPPE ANDs the two bitmasks; the 1s of the result are the matches: **0 0 1 0 1 0 0 1 0 1 1 0 1 0 0 1** = k2, k4, k7, k9, k10, k12, k15.

4.2 Two prefix sums, one fast and one slow. A full-speed prefix sum over 128 bits is large: in LoAS's own breakdown the fast one alone is 52% of a TPPE's power and 67% of its area. So LoAS uses one fast prefix sum for the weights (one offset per cycle) and a cheap laggy one for the spikes (16 adders, ready after 128/16 = 8 cycles; the scaled example also takes 8 cycles with 2 adders).

4.3 Guess, then correct. Because the spike trains are not known until the laggy prefix sum is ready, LoAS guesses that every matched input spiked at **every** timestep (1111). The weights go into one pseudo-accumulator, which reaches 10. When the laggy prefix sum is ready, LoAS reads each matched train and subtracts its weight at every timestep where the bit is 0:

timestep	**t0**	**t1**	**t2**	**t3**
pseudo-accumulator (guess: all ones)	10	10	10	10
correction (weights at the 0 bits)	5	3	5	9
result = guess - correction	**5**	**7**	**5**	**1**

Figure 5. The guess and the correction. The result matches Figure 2.

The guess costs work. The 7 matched trains have 13 ones but 15 zeros, so correcting takes more additions than adding the 1s directly would have. Real spike trains are mostly zeros: on Prosperity's real VGG16 and ResNet18 traces a non-silent train has only 2-3 ones out of 8 bits.

4.4 Cycle by cycle. Each row is a unit inside the TPPE, each column is a clock cycle. In the fast-prefix row each cell shows the match found in that cycle and the pseudo-accumulator after it.

unit \ cycle	**0**	**1**	**2**	**3**	**4**	**5**	**6**	**7**	**8**	**9**	**10**	**11**	**12**	**13**	**14**	**15**	**16**	**17**	**18**
load bitmasks + weights	bm	w																	
inner join (AND)			AND																
fast prefix sum + pseudo-accumulator				k2 3	k4 5	k7 6	k9 4	k10 7	k12 8	k15 10									
laggy prefix sum				...	...	...	...	...	...	...	rdy								
correction (fibre A read from cache, subtract)												k2 0110	k4 1111	k7 0100	k9 0011	k10 1000	k12 0101	k15 0010	last

Figure 6. LoAS on the chunk: 19 cycles. The fast prefix sum finishes at cycle 9, but the corrections cannot start until the laggy prefix sum is ready (cycle 10), and then go one per cycle. This is the cycle model of loas_model.py: load + join + (8 + matches + 1).

4.5 Between chunks. The 16 TPPEs work on 16 output pixels of the same output channel, so they share the weight chunk, which is read once and broadcast to all 16. They move to the next chunk together. A round therefore lasts as long as the slowest TPPE plus the load, and a TPPE that finished early waits. A weight chunk with no non-zeros at all is still loaded and joined.

5. Change 1 (C1): offsets without prefix sums

5.1 The idea. We need the number of 1s before a position. Instead of counting from the start of the chunk, we cut the bitmask into **segments** of 16 positions (4 in the example) and store, next to the bitmask, how many 1s come before each segment. Then:

offset = base[segment] + (number of 1s in the segment before the position)

The second term counts over at most 16 bits, which is a small circuit. The bases cost nothing at run time. The weights are fixed, so their bases are computed **offline** when the weights are packed. The spike fibres are written by LoAS's own output compressor (Section 8), which already counts 1s to pack the trains, so it writes the bases as it goes.

segment	**0 (k0-k3)**	**1 (k4-k7)**	**2 (k8-k11)**	**3 (k12-k15)**
base A: non-silent inputs before it	0	2	5	7
base B: non-zero weights before it	0	2	4	6

Figure 7. The stored bases of the chunk: a few bits per segment, next to each bitmask.

match	segment	offset in fibre A	spike train	offset in fibre B	weight
k2	0 (k0-k3)	0 + 1 = 1	0110	0 + 1 = 1	3
k4	1 (k4-k7)	2 + 0 = 2	1111	2 + 0 = 2	2
k7	1 (k4-k7)	2 + 2 = 4	0100	2 + 1 = 3	1
k9	2 (k8-k11)	5 + 0 = 5	0011	4 + 0 = 4	-2
k10	2 (k8-k11)	5 + 1 = 6	1000	4 + 1 = 5	3
k12	3 (k12-k15)	7 + 0 = 7	0101	6 + 0 = 6	1
k15	3 (k12-k15)	7 + 2 = 9	0010	6 + 2 = 8	2

Figure 8. Every match's offsets, from a base and a small count. Example: k9 is in segment 2; fibre A has 5 non-silent inputs before segment 2 and none in segment 2 before k9, so its train is at offset 5. Every offset here equals the one in Figure 4.

5.2 What disappears. Both offsets are known one cycle after the match is found, so the TPPE reads the real spike train straight away and adds the weight only at the timesteps whose bit is 1. There is nothing to guess, so there is nothing to correct: the fast and laggy prefix sums, the pseudo-accumulator, the two correction FIFOs and the correction path are all removed. Each of the T accumulators adds directly.

5.3 Cycle by cycle. The new TPPE is a 4-stage pipeline: **S0** finds the next match, **S1** computes both offsets and picks the weight, **S2** reads the spike train, and **S3** adds the weight into the timesteps whose bit is 1. A new match enters every cycle.

unit \ cycle	0	1	2	3	4	5	6	7	8	9	10	11	12
load bitmasks + bases + weights	bm	w											
start: AND			AND										
S0 find next match				k2	k4	k7	k9	k10	k12	k15			
S1 offsets + weight					k2	k4	k7	k9	k10	k12	k15		
S2 read spike train						k2	k4	k7	k9	k10	k12	k15	
S3 add into t0..t3							k2	k4	k7	k9	k10	k12	k15

Figure 9. C1 alone: 13 cycles instead of 19. The 8-cycle wait is gone. After the last match the pipeline needs 3 more cycles to drain. The accumulators end at [5,7,5,1], the same as Figure 2.

6. Change 2 (C2): four matches per cycle

6.1 The idea. With C1, finding one match per cycle is the limit. Finding several at once is now cheap, because each offset only needs its own segment. LoAS+ cuts the AND result into four **quarters** (32 positions each, 4 in the example), and each quarter gets its own match finder (a priority encoder). Every cycle, each quarter hands over its lowest remaining match.

AND bit	k0	k1	k2	k3	k4	k5	k6	k7	k8	k9	k10	k11	k12	k13	k14	k15
	0	0	1	0	1	0	0	1	0	1	1	0	1	0	0	1
found in cycle 3			X		X					X			X			
found in cycle 4								X			X					X

Figure 10. The four quarters of the AND result. Cycle 3 takes k2, k4, k9, k12 (one per quarter); cycle 4 takes k7, k10, k15. Quarter 0 has nothing left in cycle 4. A chunk takes as many cycles as its busiest quarter has matches, here 2.

6.2 Where the four weights come from. Four matches need four weights per cycle. Because weights are packed offline, the packer stores each quarter's non-zero weights in that quarter's own block of the TPPE's 128-entry weight buffer (slots 0-31, 32-63, ...). A weight then sits at *quarter x 32 + base + count*, and each of the four weight selectors chooses among 32 entries instead of 128. In the example, k15's weight sits in slot 12 + 2 = 14. The buffer is 128 entries anyway, so this layout costs no storage.

6.3 Adding four weights at once. Each timestep has its own lane, and lane t adds every arriving weight whose spike train has a 1 at t: up to 4 weights per lane per cycle (a 5-input adder with the accumulator). In cycle 6 below, lane t1 adds k2's 3, k4's 2 and k12's 1 (k9's train has a 0 at t1).

unit \ cycle	0	1	2	3	4	5	6	7
load bitmasks + bases + weights	bm	w						
start: AND			AND					
S0 find (4 quarters)				k2 k4 k9 k12	k7 k10 k15			
S1 offsets + weights					k2 k4 k9 k12	k7 k10 k15		
S2 read spike trains						k2 k4 k9 k12	k7 k10 k15	
S3 add into t0..t3							[2,6,3,1]	[5,7,5,1]

Figure 11. C1 + C2: 8 cycles instead of 19. S3 shows the accumulators after each step: [2,6,3,1] after the first four matches, then [5,7,5,1], the correct answer.

6.4 The price. Each TPPE now reads up to four spike trains per cycle, so the global cache must serve four read ports per TPPE (more banks, same capacity). A quarter that holds most of the matches limits the chunk, so in the cycle model a chunk takes its busiest quarter's match count. That is exactly what the RTL does, not the ideal matches / 4.

7. Change 3 (C3): no waiting between chunks

A layer is many chunks. In LoAS the 16 TPPEs share each weight chunk and step through the chunks together. Three things are lost there, and C3 removes each:

- **Empty weight chunks.** With 90-98% of weights pruned, some 128-position weight chunks hold no non-zero weight at all, but LoAS still loads and joins them. LoAS+ keeps one "non-empty" bit per chunk and skips empty ones entirely. If a chunk has weights but no TPPE finds a match, only the bitmask is sent and the weight values are not loaded.
- **Load, then compute.** LoAS loads a chunk, then computes it. LoAS+ double-buffers: the next chunk loads while the current one is computed.
- **Lock step.** Each TPPE gets a queue of 4 chunks and moves on as soon as it finishes. The broadcast stops only when some TPPE's queue is full.

The example below uses two TPPEs and four chunks. TPPE 0's chunks take 5, 1, 5, 1 cycles and TPPE 1's take 1, 5, 1, 5 (different pixels have different matches). Broadcasting a chunk takes 1 cycle.

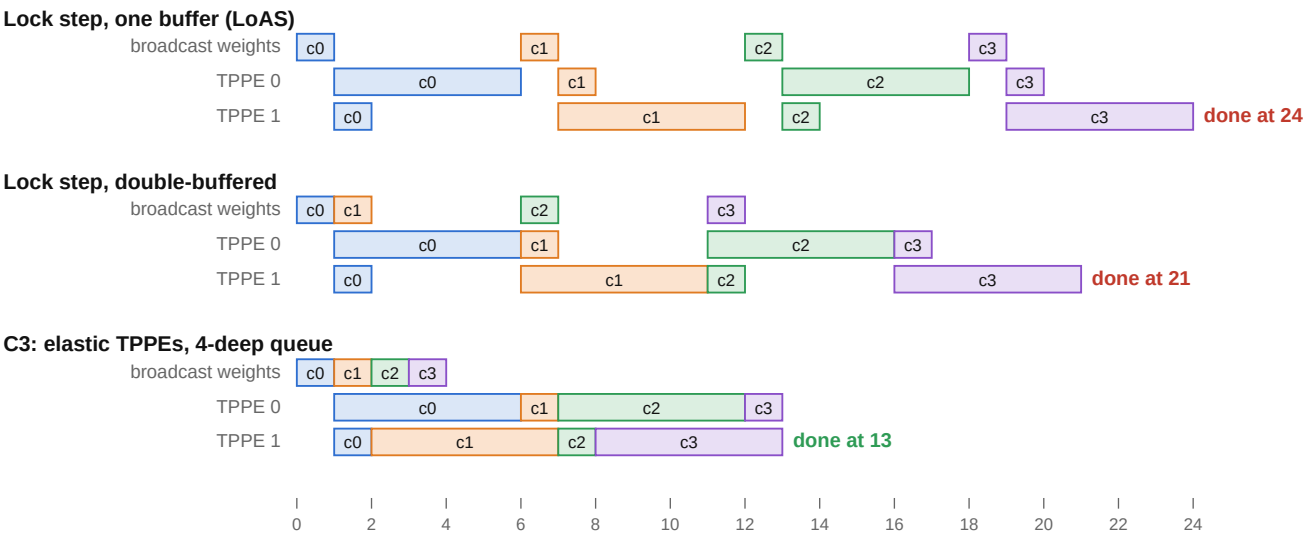


Figure 12. The same work scheduled three ways. Lock step (LoAS): every round lasts its slowest TPPE plus the load, 24 cycles. Double-buffering hides the loads: 21. Elastic queues let the short chunks fill the gaps left by the long ones: 13 cycles, the time of the busier TPPE alone. These three numbers are what loas_plus.py's scheduler computes for this case.

8. The neuron and the next layer

When an output neuron's last chunk is added, its T accumulators hold the current. In the example that is [5,7,5,1]. The P-LIF unit turns it into the spike train (0100), exactly as in LoAS (Figure 3). The compressor then writes the next layer's spike fibre. It drops silent neurons, sets the bitmask bit, appends the train, and, new in LoAS+, writes each segment's base. That base is the running count the compressor already keeps to know where the next train goes.

Extra memory. Per 128-position chunk, fibre A gets 8 bases of 7 bits (56 bits, next to its 128-bit bitmask), and fibre B gets 8 bases of 5 bits (40 bits, offsets inside a quarter). These are small next to the values: a chunk of 128 inputs with 26% non-silent holds about 33 trains x 8 bits = 266 bits.

9. The hardware, block by block

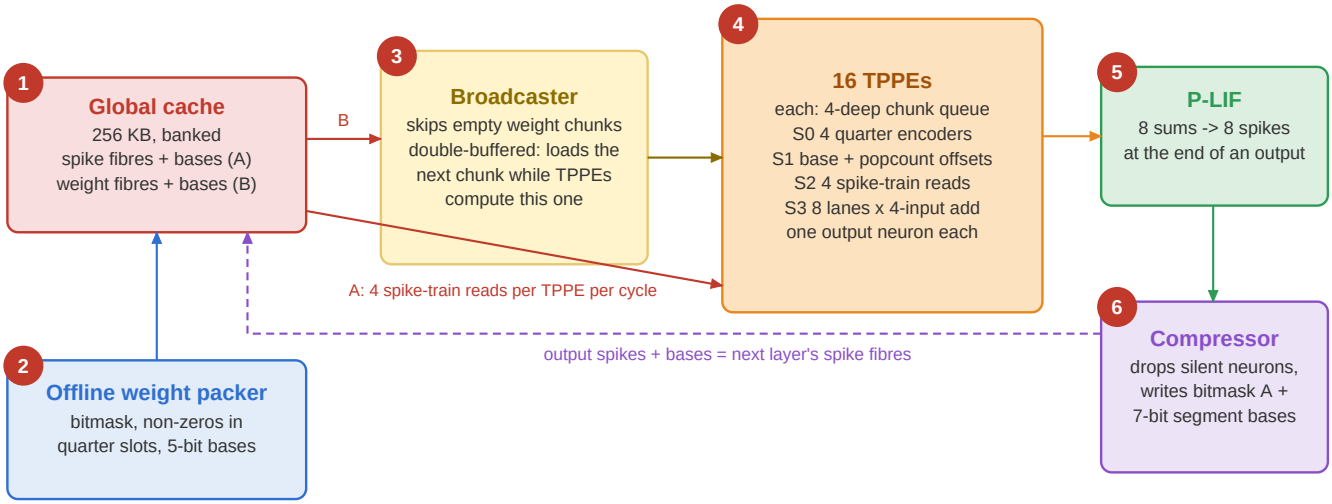


Figure 13. The blocks of LoAS+ and the order data passes through them. The numbers match the table below.

#	Block	What it does in the example
1	Global cache	Holds the fibres of Figure 4 together with their bases (Figure 7). Serves 4 spike-train reads per TPPE per cycle.
2	Offline weight packer	Builds fibre B once: the bitmask, the non-zero weights in their quarter slots, the 5-bit bases and one "chunk non-empty" bit.
3	Broadcaster	Sends each non-empty weight chunk to all 16 TPPEs and loads the next one while they compute (C3).
4	16 TPPEs	Each computes one output neuron: S0-S3 of Figure 14 on every chunk, from its own 4-deep queue (C1, C2, C3).
5	P-LIF	After an output's last chunk, turns its T sums [5,7,5,1] into spikes 0100.
6	Compressor	Packs the output spikes into the next layer's fibre A and writes its segment bases.

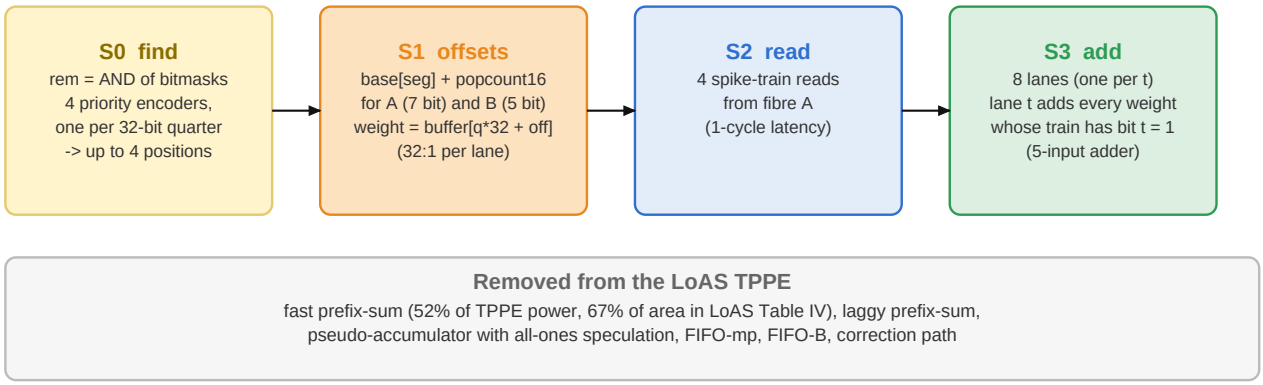


Figure 14. Inside one LoAS+ TPPE: four pipeline stages replace LoAS's prefix sums and correction path. RTL: dac2027_sim/hw/rtl/loas_plus/tppe_plus.v (bit-exact against a reference model, at RTL and gate level).

10. How much faster, and at what energy

Method. Speed comes from the same cycle model as our LoAS numbers (loas_model.py: 16 TPPEs, 128-position chunks, 16-byte weight bus), with C1-C3 added (loas_plus.py). The C1 and C2 rules are the measured behaviour of the RTL. Workloads: VGG16 and ResNet18 with synthetic tensors (90% sparse weights and spikes, and LoAS's own statistics), and Prosperity's real spike traces with 90% and 98% sparse weights. Power: each TPPE is synthesized to Nangate45 45 nm and simulated at gate level on the same stimulus as the LoAS TPPE, at 800 MHz. Energy = (16 TPPEs + 256 KB cache leakage) x latency.

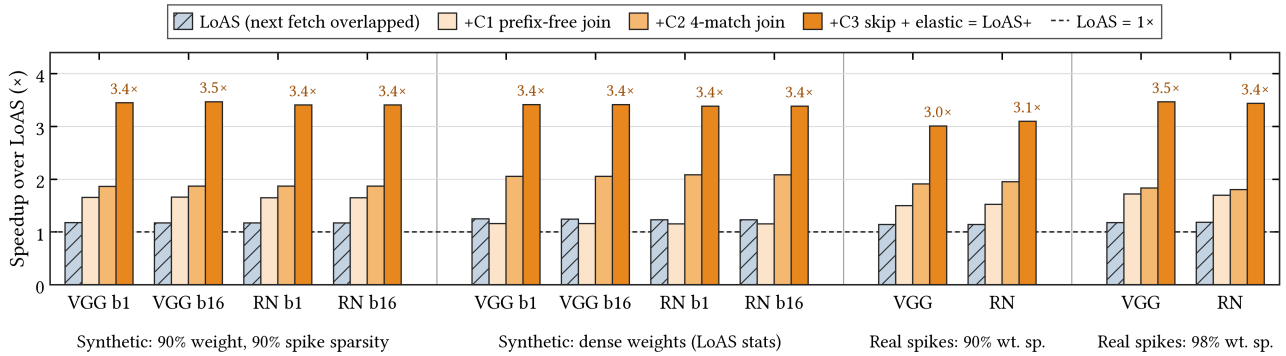


Figure 15. Speedup over LoAS, adding one change at a time. All three together: 3.4-3.5x on synthetic layers and 3.0-3.5x on real spikes, at batch 1 and 16.

	LoAS TPPE	LoAS+ TPPE
area (Nangate45)	0.0231 mm²	0.0117 mm² (-49%)
power at 800 MHz (same stimulus)	5.64 mW	6.90 mW
cycles per match (RTL, incl. the neuron)	6.3	0.77
slowest register-to-register path	3.14 ns	2.42 ns

Figure 16. One TPPE before and after.

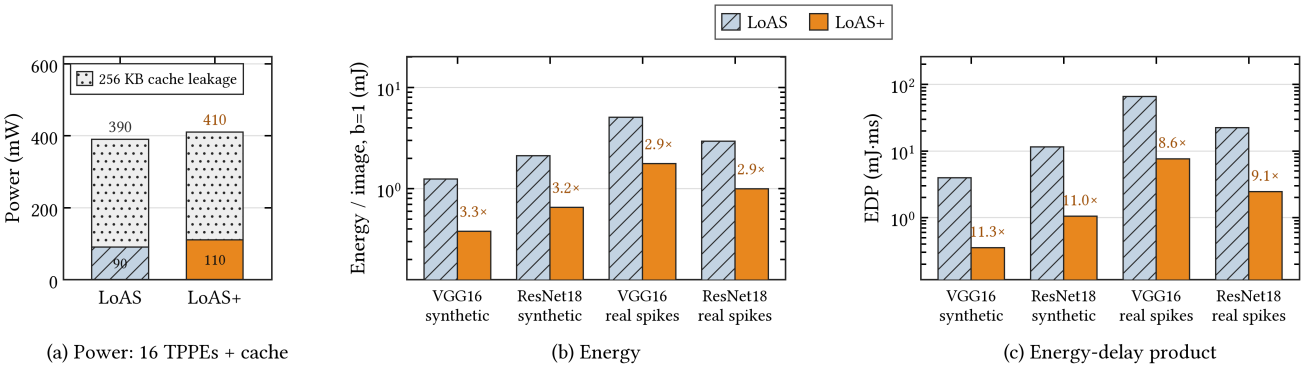


Figure 17. Power, energy per image and energy-delay product. LoAS+ draws slightly more power (410 vs 390 mW with the cache) but finishes about 3.4x sooner. That gives 2.9-3.3x less energy and 8.6-11.3x lower EDP.

Both comparisons favour LoAS. The LoAS TPPE RTL we measure fails its own reference check and runs slowly (6.3 cycles per match), so its measured power reflects low activity. The LoAS+ TPPE is measured while it is busy every cycle.

11. How this differs from LoAS

	LoAS	LoAS+
finding a match's offset	prefix sum over 128 bits (one fast, one laggy)	stored base per 16-bit segment + count over 16 bits
spike train of a match	guessed as all ones, corrected 8+ cycles later	read directly, 1 cycle after the match
matches per cycle	1	up to 4 (one per 32-bit quarter)
additions per match	one guess + one correction per 0 bit	one per 1 bit
empty weight chunk	loaded and joined	skipped
next chunk's load	after the current chunk	during the current chunk
16 TPPEs	lock step, wait for the slowest	each from its own 4-deep queue
extra stored data	-	56 bits (spikes) + 40 bits (weights) per 128-chunk
cache read ports per TPPE	1	4

Figure 18. LoAS and LoAS+, part by part.

The dataflow (inner product, all timesteps in parallel, one output neuron per TPPE), the compression format, the neuron and the results are all LoAS's. LoAS+ changes how a TPPE walks through a chunk and how the TPPEs walk through the chunks.

12. What it costs, and what is still open

- **Metadata:** 96 bits of bases per 128-position chunk (Section 8). The memory footprint grows slightly; this is not yet counted in the traffic model.
- **Cache ports:** four spike-train reads per TPPE per cycle. The 256 KB cache needs 4x the banks or ports. Its leakage is counted, but cache read/write energy is not counted for either design.
- **Pipeline drain:** the TPPE drains its 3-stage pipeline after every chunk. Chunks of the same output neuron add into the same accumulators, so the drain is only needed once per neuron. Removing it should take VGG16 from 3.4x to about 5.9x. This is the next RTL change.
- **Clock:** neither TPPE meets 800 MHz in synthesis without placement, so both are compared at the same clock. The LoAS+ datapath is the shorter of the two.
- **Baseline:** the LoAS TPPE RTL we have fails its own testbench; a corrected LoAS RTL would draw more power and make LoAS+ look better, not worse.
- **Open:** real trained weights (the weights here are synthetic at a given sparsity), a full-chip area comparison, and the ideas not yet modelled: keeping each TPPE's spike fibre local (cache energy), and skipping the negative weights of outputs that provably cannot spike.